

WEEK 12 ASSIGNMENT: BUILDING THE RAG CHATBOT ENGINE & USER INTERFACE

- **Objective:** Integrate the Vector Database built in Week 11 with a Large Language Model (LLM) to complete a functional Retrieval-Augmented Generation (RAG) pipeline, and deploy a user-friendly chat interface for corporate internal use.

PART 1: ROLE-BASED TASK DISTRIBUTION

To ensure professional agile project management and fair grading, tasks are strictly divided among project roles. Each member must commit specific artifacts to GitHub that reflect their responsibilities:

1.1. Project Manager (PM) – Governance & Scope Management

- **User Story Mapping:** Define at least 3 business user stories for the chatbot (e.g., "As an HR Manager, I want the chatbot to query the internal handbook so that I can onboard employees faster"). Document these in `evaluation_metrics.md`.
- **Guardrails & Risk Mitigation:** Establish system guardrails. Define explicit boundaries on what the chatbot **is not allowed** to answer (e.g., competitor comparisons, internal salaries, or restricted financial data).
- **Release Coordination:** Ensure the `.env.example` is properly documented, the `.gitignore` is active, and coordinate the final repository structure before submission.

1.2. Business Analyst (BA) – Prompt Engineering & Analytics

- **Persona & System Prompt Design:** Author the master system prompt template. Write the exact instruction set that shapes the AI's persona, forcing it to remain factual and rely *only* on the corporate context extracted from Week 11.
- **Exception & Fallback Handling:** Define the exact logic for edge cases (e.g., when the retrieved data from the Vector DB is insufficient or irrelevant, what safe fallback answer should the chatbot provide?).
- **System Evaluation:** Draft the analytical review in `evaluation_metrics.md`, documenting how the chatbot performed during group testing against typical user queries.

1.3. Developer (Dev) – RAG Core & Interface Engineering

- **RAG Engine Implementation (`rag_engine.py`):** Code the technical pipeline. Develop functions to capture user input, generate its embedding, query the existing Vector DB, extract the top-k context chunks, and merge them into the BA's prompt template before sending the payload to the LLM API (OpenAI, Gemini, or local Ollama).
- **UI Application (`app.py`):** Build a web-based, conversational user interface using **Streamlit** or **Gradio**. Connect the chat box inputs directly to the `rag_engine.py` functions and implement a sidebar for model parameter configurations (e.g., Top-K value, temperature).
- **Dependency Management:** Maintain and clean the `requirements.txt` file ensuring all external libraries are runnable.

PART 2: TECHNICAL DELIVERABLES

The code repository must functionally execute the tasks divided above:

- **rag_engine.py:** A Python script capable of taking a user prompt string, retrieving top-k context vectors, merging them into the system prompt, and returning the LLM's text output.
- **app.py:** A running application featuring a clean conversational chat UI, system parameter sidebar, and an explicit warning if the system encounters a fallback scenario.

PART 3: CRITICAL THINKING & EVALUATION QUESTION

Discuss as a group and append your answers to the bottom of your `evaluation_metrics.md` file:

Question: > *"During testing, your chatbot might encounter a '**Hallucination**' (generating realistic but false information) or a '**Context Overflow**' (retrieving too much text, exceeding the LLM's prompt token limit). Based on your Week 12 implementation, propose two distinct strategies to mitigate Hallucinations and optimize the Context Window size for your enterprise data."*

APPENDIX: AI-ASSISTED LEARNING FRAMEWORK (MANDATORY GUIDELINES)

Students are encouraged to use generative AI (ChatGPT, Gemini, Claude) as a learning catalyst. However, **blind copy-pasting is strictly prohibited**. Your grading will heavily rely on your understanding during the oral defense (viva).

The "Pilot & Co-Pilot" Protocol

- **AI is your Co-Pilot:** Use AI to explain complex errors, suggest library functions (e.g., Streamlit session states), or brainstorm prompt structures.
- **You are the Pilot:** You must understand every line of code committed under your role. If a student cannot explain the code or documentation generated by an AI during evaluation, **the entire group will face a severe grade deduction**.

Approved Prompt Patterns for This Assignment

Instead of asking AI to *"write the whole project"*, use targeted educational prompts:

- **For BA (Prompt Engineering):** *"Act as an expert prompt engineer. I need a system prompt for a corporate RAG chatbot that strictly forbids the AI from making up information. Give me a structured template."*
- **For Dev (Debugging):** *"I am getting a KeyError: 'st.session_state' in my Streamlit app when resetting the chat history. Here is my code block. Explain why this happens and how to fix it."*

SUBMISSION REQUIREMENTS

The evaluation of Week 12 relies on a dual-platform submission workflow. Teams must submit their repository link via MLearning's "Online Text" feature to lock their deadline, while the technical grading and code execution will be conducted directly from their GitHub Repository.

Submission Workflow & Deadlines

1. **STEP 1 (Code Push):** All source code files, updated environment templates, and system evaluation files must be committed and pushed to the team's GitHub Repository before the official deadline.
2. **STEP 2 (MLearning Online Text Submission):** Instead of uploading files, the Team Leader (PM) must log into MLearning, open the assignment, and paste the required information directly into the **Online Text** submission box using the following template:

Plaintext

TEAM NAME: KMS_TEAM_XX

GITHUB REPOSITORY URL: [Paste your active GitHub link here]

ROSTER & CONTRIBUTION BREAKDOWN:

- Student A (Project Manager) - Student ID - XX% Contribution
- Student B (Business Analyst)- Student ID - XX% Contribution
- Student C (Developer) - Student ID - XX% Contribution

Required Directory Structure (Inside GitHub)

Your local development via VS Code must be pushed to your repository keeping the root directory clean, unpolluted by virtual environment folders (.venv), and structured as follows:

Plaintext

KMS_TEAM_XX_W12/

```
|
|--- .gitignore
|--- .env.example
|--- requirements.txt      <-- Maintained by DEV (includes streamlit, openai/google-generativeai)
|
|--- ingest_to_vector.py   <-- Retained from W11
|--- rag_engine.py        <-- NEW: Built by DEV (Core RAG processing logic)
|--- app.py               <-- NEW: Built by DEV (Streamlit/Gradio Web UI Application)
|--- evaluation_metrics.md <-- NEW: Written by BA & PM (Contains User Stories, Guardrails, & Part 3 answers)
```

CRITICAL WARNING: NEVER commit your actual .env file containing private API Keys (OpenAI, Gemini, or Database Passwords) to GitHub. Any repository exposing active API keys will receive an automatic **zero (0) grade** for the entire week due to security violations. Ensure your keys are only stored locally and .env is listed in your .gitignore.